

Work2 ウェブアプリの作成

このワークでは、独自のトークンを作成し、それを誰かに送付することを行います。

<http://localhost:3000/tokens> で表示される画面の機能を実装していきます。

※Codespaces 利用者は、<http://localhost:3000> の代わりに自分の Codespaces が割り当てた URL を使用してください。
Codespaces は以下のように URL を生成します。
<https://<ユーザー名>-<リポジトリ名>-<ランダムID>-<ポート>.app.github.dev/>

現時点では、<http://localhost:3000/tokens> にアクセスするとエラー画面が表示されます。

安心してください。これからきちんと表示されるように実装を進めていきます。

1. トークンの新規作成と送付

1.1. 保持トークンの一覧表示

最初は、自分が保有しているトークンを表示できるようにします。そのために、**TapyrusAPI**のトークンの総量取得 API を実装しましょう。**TapyrusApi** クラスの **get_tokens** メソッドを実装します。

以下の通り実装することで、TapyrusAPI のトークンの総量取得 API を実行できます。

編集対象のファイルは **lib/utils/tapyrus_api.rb** です。

```
def get_tokens(confirmation_only = true)
  res = instance.connection.get("/api/v2/tokens") do |req|
    req.headers['Authorization'] = "Bearer #{instance.access_token}"
    req.params['confirmation_only'] = confirmation_only
  end

  res.body[:tokens]
end
```

これらのコードは、TapyrusAPI の次の機能呼び出しています。

<https://doc.api.tapyrus.chaintope.com/#operation/getTokensV2>

実装が完了したら、<http://localhost:3000/tokens> にアクセスしてみましょう。現在保有しているトークンと、保有量が表示されます。前回のハンズオンで作成したトークンが表示されると思います。

1.2. トークンの新規発行

続いて、トークンの新規発行 API を実装しましょう。

トークンの新規発行も TapyrusAPI を使うことで簡単に行えます。**TapyrusApi** クラスの **post_tokens_issue** メソッドを以下の通り実装することで、TapyrusAPI のトークンの新規発行 API を実行できます。

編集対象のファイルは **lib/utils/tapyrus_api.rb** です。

```
def post_tokens_issue(amount:, token_type: 1, split: 1)
  res = instance.connection.post("/api/v2/tokens/issue") do |req|
    req.headers['Authorization'] = "Bearer #{instance.access_token}"
    req.headers['Content-Type'] = 'application/json'
    req.body = JSON.generate({ "amount" => amount, "token_type" =>
token_type, "split" => split })
  end

  res.body
end
```

TapyrusAPI は REST API なので、最初の `res =`

`instance.connection.post("/api/v1/tokens/issue") do |req|` で TapyrusAPI のトークンの新規発行のエンドポイントを呼び出しています。

次の行の `req.headers['Authorization'] = "Bearer #{instance.access_token}"` は TapyrusAPI へアクセスするためのアクセストークンを指定しています。

TapyrusAPI ではアクセストークン毎に `wallet` が作成されていますので、これはトークンを新規発行する対象の `wallet` を指定していることでもあります。

次の 2 行では、トークンの新規発行のエンドポイントに必要なパラメータを指定しています。

```
req.headers['Content-Type'] = 'application/json'
req.body = JSON.generate({ "amount" => amount, "token_type" =>
token_type, "split" => split })
```

これらのコードは、TapyrusAPI の次の機能を呼び出しています。

<https://doc.api.tapyrus.chaintope.com/#operation/issueTokenV2>

ドキュメントにも記載がある通り、TapyrusAPI では以下の 3 種類のトークンが発行可能です。

1. 再発行可能なトークン
2. 再発行不可能なトークン
3. NFT

実装したら早速ブラウザで確認します。

<http://localhost:3000/tokens/new> にアクセスします。

今回は、再発行可能トークンを 100 発行します。次の通り指定して実行しましょう。 `Amount` には `100` を、 `Token Type` には `再発行可能トークン` を、 `Split` には `10` を入力して `BCに記録` を押してみましょう。

しばらく（約10分ほど）するとトークンが新規発行され、トークン一覧画面(<http://localhost:3000/tokens>)に表示されます。

1.3. アドレス一覧の表示

作成したトークンを早速送付したいのですが、トークンを送付するためには受け取りアドレスが必要です。なので、次はアドレス一覧の表示及び、新規作成機能を実装します。

まずはアドレス一覧の表示機能から実装します。 `http://localhost:3000/wallets` でアドレスの一覧が表示されるように実装します。

現時点ではアクセスするとエラー画面が表示されますが、`TapyrusApi` クラスの `get_addresses` メソッドを実装することで該当画面にアドレス一覧が表示されるようになります。

以下のコードを実装して下さい。

```
def get_addresses(per: 25, page: 1, purpose: "general")
  res = instance.connection.get("/api/v1/addresses") do |req|
    req.headers['Authorization'] = "Bearer #{instance.access_token}"
    req.params['per'] = per
    req.params['page'] = page
    req.params['purpose'] = purpose
  end

  res.body
end
```

これらのコードは、TapyrusAPI の次の機能呼び出しています。

<https://doc.api.tapyrus.chaintope.com/#operation/getAddresses>

実装が完了したら、`http://localhost:3000/wallets` にアクセスしてみましょう。自分のwalletで管理しているアドレスの一覧が表示されます。

1.4. アドレスの新規作成

続いてアドレスの新規作成機能を実装します。walletを初めて使用するときはまだアドレスが存在していません。また、用途に応じてアドレスを使い分けたい場合もあると思います。

アドレスの新規作成機能を追加するために、`TapyrusApi` クラスの `post_addresses` メソッドを実装しましょう。

以下の通り実装することで、TapyrusAPI のアドレスの生成 API を実行できます。

```
def post_addresses(purpose: "general")
  res = instance.connection.post("/api/v1/addresses") do |req|
    req.headers['Authorization'] = "Bearer #{instance.access_token}"
    req.headers['Content-Type'] = 'application/json'
    req.body = JSON.generate({ "purpose" => purpose })
  end

  res.body
end
```

これらのコードは、TapyrusAPI の次の機能呼び出しています。
<https://doc.api.tapyrus.chaintope.com/#operation/createAddress>

アドレス作成 を押すと 1 つアドレスが新規に作成されます。

このアドレスをトークンの送り主に教えましょう。Discordにご自身で発行したアドレスをメッセージ投稿してください。

1.5 トークンの送付

最後に作成したトークンを誰かに送ってみましょう。

TapyrusApi クラスの **put_tokens_transfer** メソッドを実装しましょう。以下の通り実装することで、TapyrusAPI のトークンの送付 API を実行できます。

編集対象のファイルは **lib/utils/tapyrus_api.rb** です。

```
def put_tokens_transfer(token_id, address:, amount:)
  res = instance.connection.put("/api/v2/tokens/#{token_id}/transfer") do
    |req|
      req.headers['Authorization'] = "Bearer #{instance.access_token}"
      req.headers['Content-Type'] = 'application/json'
      req.body = JSON.generate({ "address" => address, "amount" => amount })
    end

    res.body
  end
end
```

これらのコードは、TapyrusAPI の次の機能呼び出しています。
<https://doc.api.tapyrus.chaintope.com/#operation/transferTokenV2>

実装が完了したら、実際に誰かにトークンを送ってみましょう。

<http://localhost:3000/tokens/transfer> にアクセスします。

自分が所有している送りたいトークンの **Token Id** を指定し、送り先の **Address** と送付量 **Amount** を入力して **BCに記録** しましょう。

トークンの送付はできましたか？

最後に、誰かから送られてきたトークンがどれだけあるのか確認したいですね。

<http://localhost:3000/tokens> にアクセスして確認してみましょう。

いかがでしたか？

自分で作成したトークン以外のトークンが表示されましたか？

以上でトークンの送付と確認のワークは終了になります。